

Ejercicio 3 — Análisis Teórico

Trabajo Práctico 1: Interrupciones y ADC por Polling

Sistemas de Tiempo Real

1. Parámetros Temporales y Frecuencias

1.1. Prescaler del ADC

El datasheet del ATmega328P especifica que el reloj del ADC (F_{ADC}) debe estar comprendido entre 50 kHz y 200 kHz para garantizar los 10 bits de resolución. Con un cristal de 16 MHz (F_{CPU}) se selecciona un prescaler de 128 (ADPS2:0 = 111):

$$F_{ADC} = \frac{F_{CPU}}{\text{Prescaler}} = \frac{16\,000\,000}{128} = 125\text{ kHz}$$

El valor de 125 kHz se ubica en el centro del rango admisible, maximizando el margen de tolerancia frente a variaciones de tensión o temperatura.

1.2. Tiempo de Conversión

Una conversión normal (posterior a la primera) toma exactamente 13 ciclos de reloj del ADC:

$$T_{conv} = 13 \cdot \frac{1}{F_{ADC}} = 13 \cdot \frac{1}{125\,000} = 104\text{ }\mu\text{s}$$

Durante este intervalo la CPU permanece en espera activa (*polling*), bloqueada hasta que el ADC levante la bandera ADIF.

1.3. Resolución Analógica

Con referencia de tensión $V_{REF} = 5\text{ V}$ y convertidor de $n = 10$ bits:

$$\text{Resolución} = \frac{V_{REF}}{2^n} = \frac{5\text{ V}}{1024} \approx 4,88\text{ mV por bit}$$

2. Análisis del Código

2.1. Polling de bandera (adc_read)

El enunciado requiere explícitamente *consulta de bandera (polling)*. La implementación utiliza espera activa:

```

1 while (!(ADCSRA & (1 << ADIF)));
2 ADCSRA |= (1 << ADIF);

```

La espera activa es aceptable en este contexto porque el sistema es *bare-metal* de lazo único: no existe otra tarea bloqueada durante los 104 µs de conversión. El comportamiento es completamente determinista.

La bandera ADIF sigue la convención **Clear-on-Write-1** de la arquitectura AVR: se limpia escribiendo un 1 lógico sobre el bit.

2.2. Atomicidad en la lectura (adc_init y lectura)

Al configurar ADLAR = 0 mediante:

```

1 ADMUX = (1 << REFS0);

```

los 10 bits de resultado quedan alineados a la derecha en los registros ADCL/ADCH. La instrucción `return ADC;` garantiza atomicidad: el compilador AVR-GCC lee primero ADCL (lo que bloquea por hardware la actualización del par de registros) y luego ADCH, asegurando la integridad del dato de 16 bits.

3. Multiplexado y Tiempos de Latching

3.1. Manipulación a nivel de bits

Para separar los nibbles destinados a cada latch 74LS373:

```

1 uint8_t byte_alto = (uint8_t)((valor >> 8) & 0x0F);
2 uint8_t byte_bajo = (uint8_t)(valor & 0xFF);

```

- **Desplazamiento (8)**: mueve los dos bits más significativos del resultado de 10 bits a las posiciones 0 y 1.

Enmascaramiento (& 0x0F): fuerza a cero los cuatro bits altos del byte, evitando activar pines no deseados del PORTB conectados a hardware externo.

3.2. Tiempos de latch (Setup y Hold Time)

```

1 PORTC |= (1 << PORTC2);    // LE alto
2 _delay_us(5);
3 PORTC &= ~(1 << PORTC2);  // LE bajo

```

El retardo de 5 µs no es arbitrario: cumple el *Pulse Width* mínimo y el *Data Setup Time* especificados en el datasheet del 74LS373. Si el microcontrolador conmutara la señal LE a la frecuencia del sistema ($\approx 62,5$ ns por ciclo), las compuertas TTL no alcanzarían a estabilizarse, dando lugar a capturas erróneas por capacitancia parásita.

4. Período de Muestreo

El lazo principal incluye `_delay_ms(50)`. El período real de cada iteración es la suma de todas las operaciones:

$$T_{ciclo} = T_{conv}(104 \mu s) + T_{latch alto}(5 \mu s) + T_{latch bajo}(5 \mu s) + 50 \text{ ms} + t_{overhead}$$

El retardo de 50 ms fija una tasa de refresco de 20 Hz, suficiente para la persistencia visual del ojo humano sin generar *aliasing* perceptible en el display de 7 segmentos.

La implementación mediante `while(1)` con retardo bloqueante introduce *jitter* inherente. Para un sistema de tiempo real estricto se debería emplear un temporizador por interrupción; sin embargo, el enunciado requiere explícitamente el modo de muestreo por bucle y consulta de bandera, y la tolerancia de una interfaz humana acepta dicha fluctuación.